TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT HƯNG YÊN

KHOA CÔNG NGHỆ THÔNG TIN

--------- oOo ---------



BÀI TẬP THỰC HÀNH

TRUYỀN THÔNG TRONG IoT

TRÌNH ĐỘ: ĐẠI HỌC

NGÀNH ĐÀO TẠO: CÔNG NGHỆ THÔNG TIN

Hưng Yên – Tháng 01 năm 2025

BÀI THỰC HÀNH SỐ 12: GIAO THỨC TẦNG ỨNG DỤNG - MQTT

A. MỤC TIÊU BÀI THỰC HÀNH

Sau bài thực hành này sinh viên có thể:

- Trình bày được nguyên lý hoạt động của giao thức MQTT và vai trò của MQTT trong hệ thống truyền thông IoT.
- Phân tích được mô hình publish/subscribe và vai trò của các thành phần: thiết bị IoT (ESP), MQTT broker và ứng dụng (Home Assistant).
- Thực hành truyền dữ liệu, điều khiển thiết bị và xây dựng hệ thống tự động hóa IoT dựa trên MQTT.
- Hiểu và vận dụng các khái niệm quan trọng của MQTT như topic, payload, retain message và mô hình xử lý hướng sự kiện (event-driven).

B. ĐIỀU KIỆN THỰC HÀNH

Để thực hiện vài thực hành này, sinh viên cần:

1. Kiến thức yêu cầu
   - Mô hình OSI/TCP-IP và vai trò của tầng ứng dụng.
   - Mạng máy tính: địa chỉ IP, giao thức TCP/IP.
   - Nguyên lý hoạt động của vi điều khiển ESP32/ESP8266.
   - Ngôn ngữ C/C++ (Arduino).
2. Phần cứng
   - 01 bộ kit ESP32 hoặc ESP8266 cho mỗi sinh viên/nhóm.
   - 01 LED và điện trở (hoặc sử dụng LED tích hợp trên kit).
   - 01 nút nhấn (push button).
   - Cáp USB kết nối ESP với máy tính.

- Hệ thống mạng LAN/WiFi dùng chung cho ESP, máy tính và Home Assistant

3. Phần mềm

- Arduino IDE hoặc PlatformIO để lập trình ESP.
- Home Assistant (cài đặt dạng Supervised hoặc tương đương).
- Mosquitto MQTT Broker (Add-on cho Home Assistant).
- Wireshark (phục vụ phân tích gói tin, khuyến nghị).
- Trình duyệt web để truy cập giao diện Home Assistant.
- (Tùy chọn) MQTT Explorer hoặc công cụ dòng lệnh mosquitto_pub, mosquitto_sub để kiểm tra topic.

C. TÀI NGUYÊN THAM CHIẾU

Để hoàn thành tốt bài thực hành này sinh viên nên tham khảo các tài nguyên sau:

STT	Tên tài nguyên	Mô tả tài nguyên
1	Bai tap thuc hanh 12.pdf	Tài liệu bài thực hành số 12
2	Các bài giảng về MQTT	Slide bài giảng về MQTT

D. YÊU CẦU BÀI THỰC HÀNH

Bài 12.1. Truyền thông MQTT giữa ESP và Home Assistant

Bài 12.2. Điều khiển thiết bị IoT từ Home Assistant qua MQTT

Bài 12.3. Giữ trạng thái thiết bị IoT: Retain và bài toán "nhớ/quên"

E. HƯỚNG DẪN THỰC HIỆN

Bài 12.1. Truyền thông MQTT giữa ESP và Home Assistant

Mục tiêu

Bài thực hành này giúp sinh viên quan sát và phân tích trực tiếp hoạt động của giao thức MQTT trong một hệ thống IoT thực tế sử dụng ESP và Home Assistant. Thông qua việc bắt

và phân tích gói tin bằng Wireshark, sinh viên hiểu rõ cơ chế publish/subscribe, vai trò của broker và mối quan hệ giữa các MQTT client.

Chuẩn bị

- *Phần cứng*
  - Module wifi ESP32/8266
  - Cáp USB
- *Phần mềm*
  - Arduino IDE / PlatformIO
  - Thư viện PubSubClient
  - Home Assistant + Mosquitto Broker
  - Wireshark

Lưu ý: ESP, HA, máy bắt Wireshark **cùng LAN**

Gợi ý thực hiện

Bước 1 – ESP publish dữ liệu

- ESP kết nối Wifi
- Publish JSON

```
// IMPORT IF ESP32
// #include <WiFi.h>
// IMPORT IF ESP8266
#include <ESP8266WiFi.h>


#include <PubSubClient.h>
#include <ArduinoJson.h>
```

```
/* ====== CONFIG ====== */
const char* device_name = "YOUR_DEVICE_NAME";

const char* ssid = "YOUR_WIFI_SSID";
const char* password = "YOUR_WIFI_PASSWORD";

// IP Home Assistant
const char* mqtt_server = "HA_IP_ADDRESS";
const int   mqtt_port   = 1883;

const char* mqtt_user = "MQTT_USERNAME";
const char* mqtt_pass = "MQTT_PASSWORD";

const char* student_id  = "MASV";
const char* mqtt_topic  = "iot/lab1/MASV/sensor";

/* ==================== */

WiFiClient espClient;
PubSubClient client(espClient);
```

```
void setup_wifi() {
  delay(10);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
  }
}

void reconnect() {
  while (!client.connected()) {
    String clientId = device_name;
    clientId += student_id;

    if (client.connect(clientId.c_str(), mqtt_user, mqtt_pass)) {
      Serial.println("MQTT connected");
    } else {
      Serial.print("MQTT failed, rc=");
      Serial.println(client.state());
      delay(2000);
    }
  }
}
```

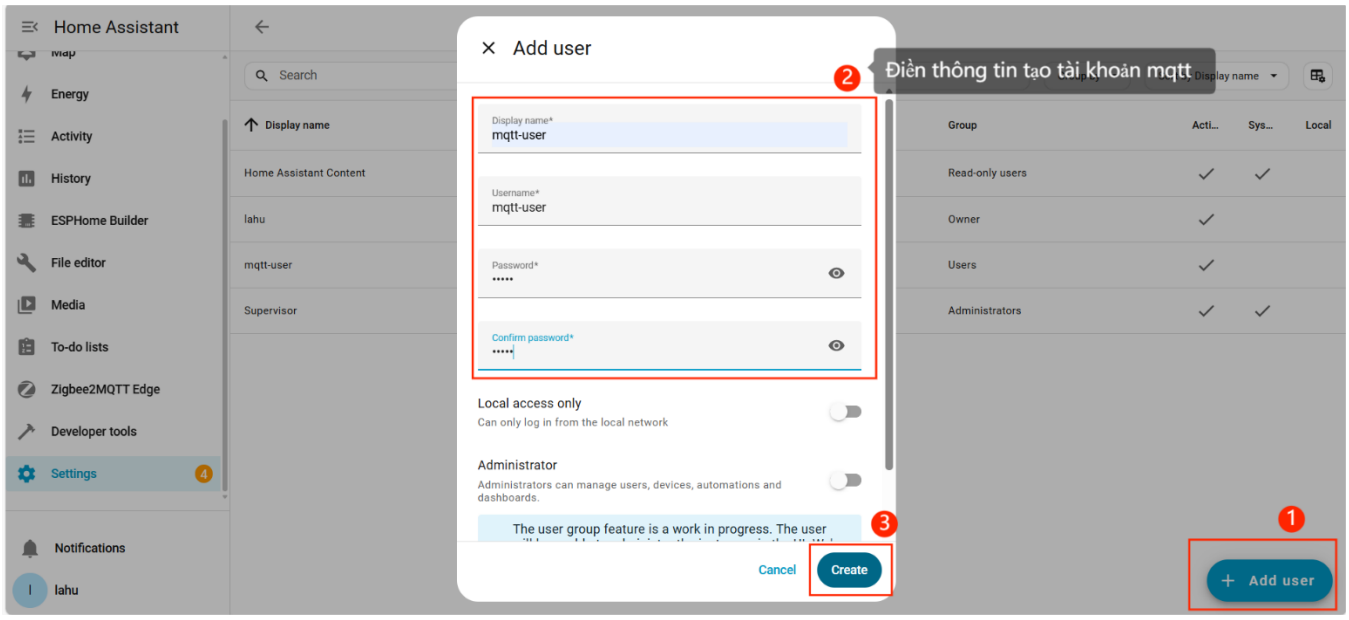
```
void setup() {
  Serial.begin(115200);
  setup_wifi();
  client.setServer(mqtt_server, mqtt_port);
}

void loop() {
  if (!client.connected()) {
    reconnect();
  }
  client.loop();
  StaticJsonDocument<200> doc;
  doc["temp"] = random(250, 350) / 10.0;
  doc["hum"]  = random(400, 800) / 10.0;
  char payload[128];
  serializeJson(doc, payload);
  client.publish(mqtt_topic, payload);

  delay(5000);
}
```

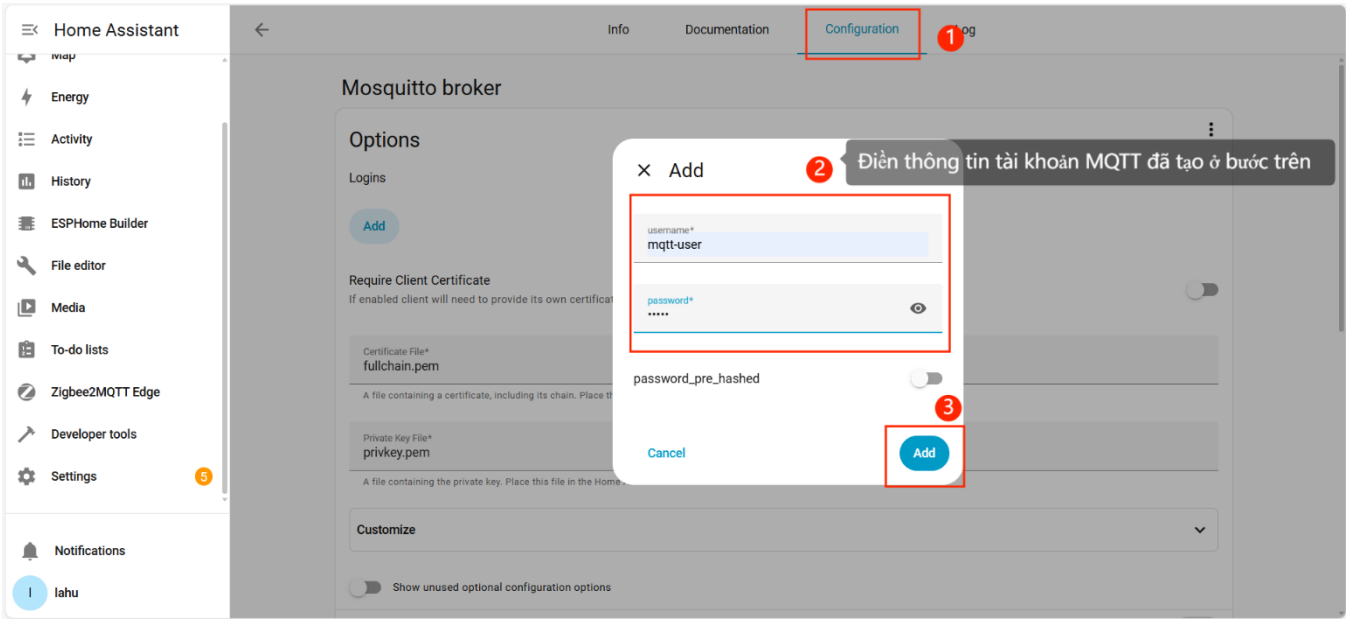
Bước 2 – Home Assistant subscribe

- Tạo tài khoản MQTT trên HA: **Settings > People > Uses > Add user**



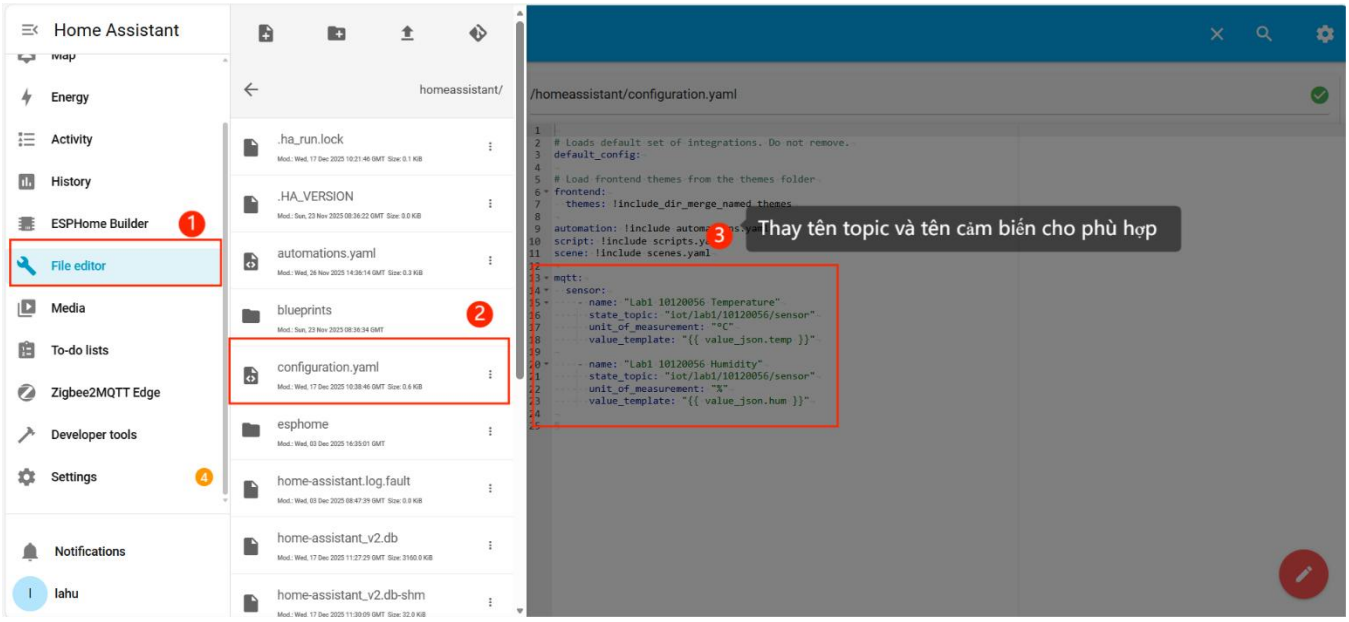
Hình 8.1. Tạo tài khoản người dùng MQTT trên Home Assisant

- Đăng ký tài khoản MQTT với Broker: **Settings > Add-ons > Mosquitto broker**



Hình 8.2. Đăng ký tài khoản MQTT với MQTT Broker

- Tạo MQTT Sensor trong HA

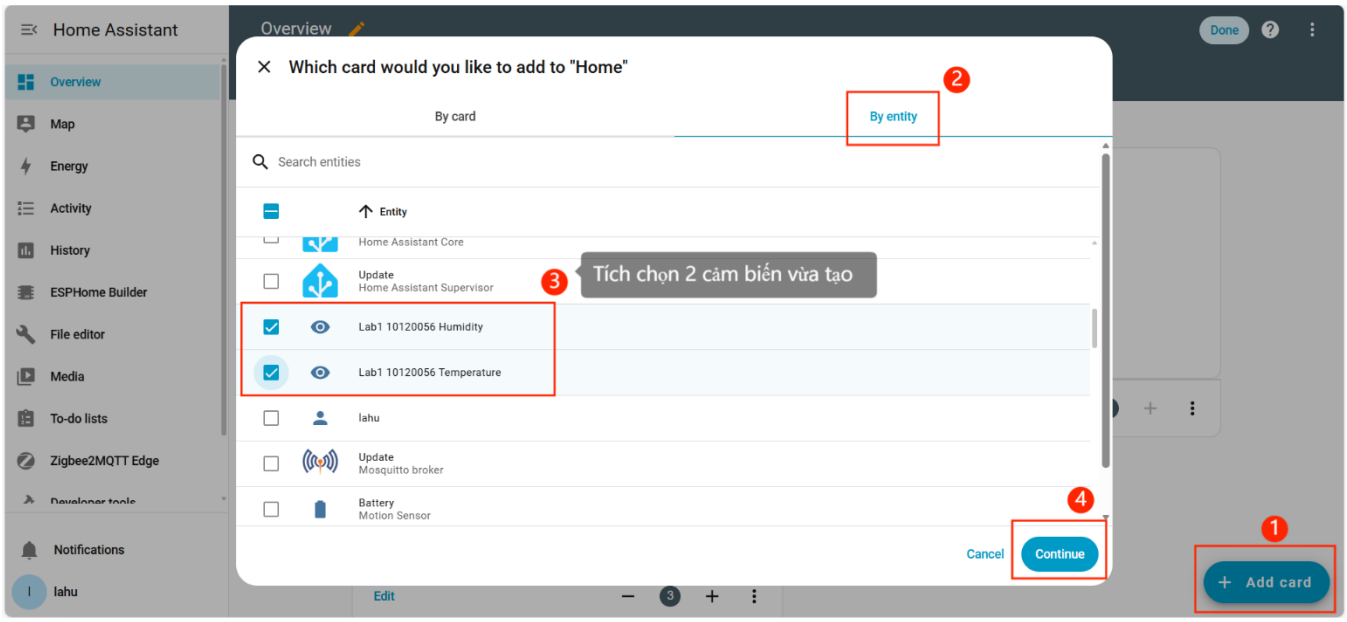


Hình 8.3. Tạo MQTT Sensor trên Home Assistant

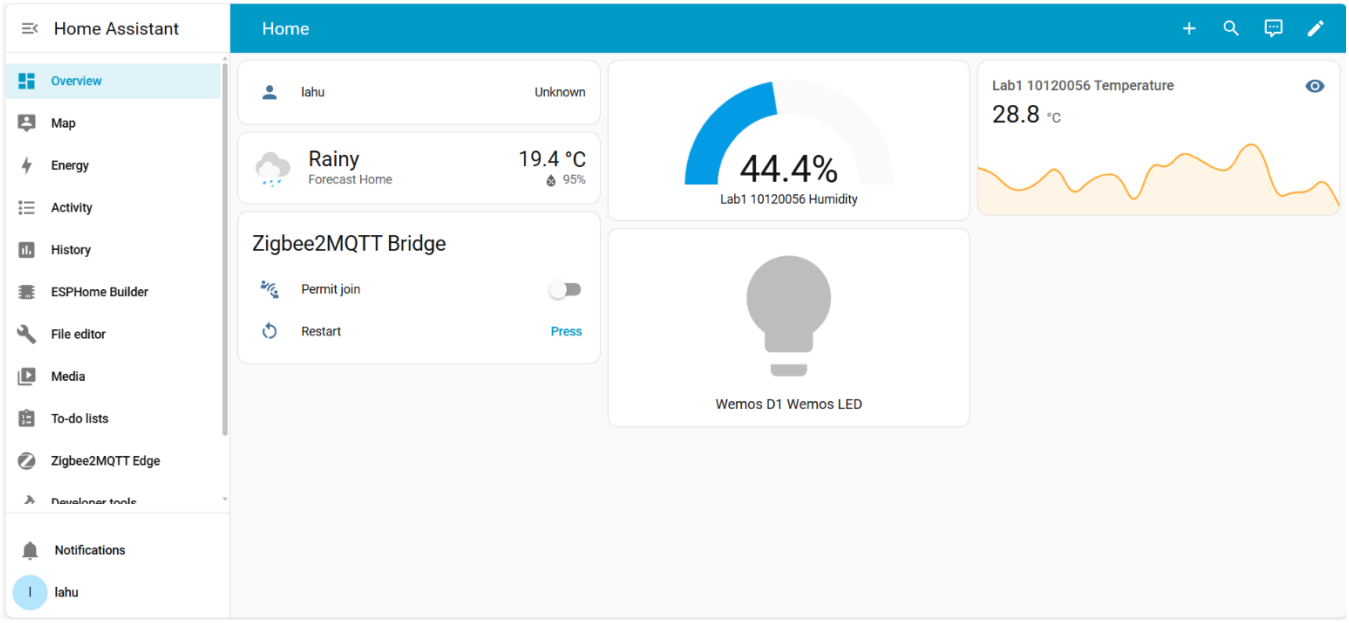
```
mqtt:
  sensor:
    - name: "Lab1 10120056 Temperature"
      state_topic: "iot/lab1/10120056/sensor"
      unit_of_measurement: "°C"
      value_template: "{{ value_json.temp }}"

    - name: "Lab1 10120056 Humidity"
      state_topic: "iot/lab1/10120056/sensor"
      unit_of_measurement: "%"
      value_template: "{{ value_json.hum }}"
```

- Hiển thị nhiệt độ, độ ẩm: **Overview > Edit dashboard > Add card > By entity**



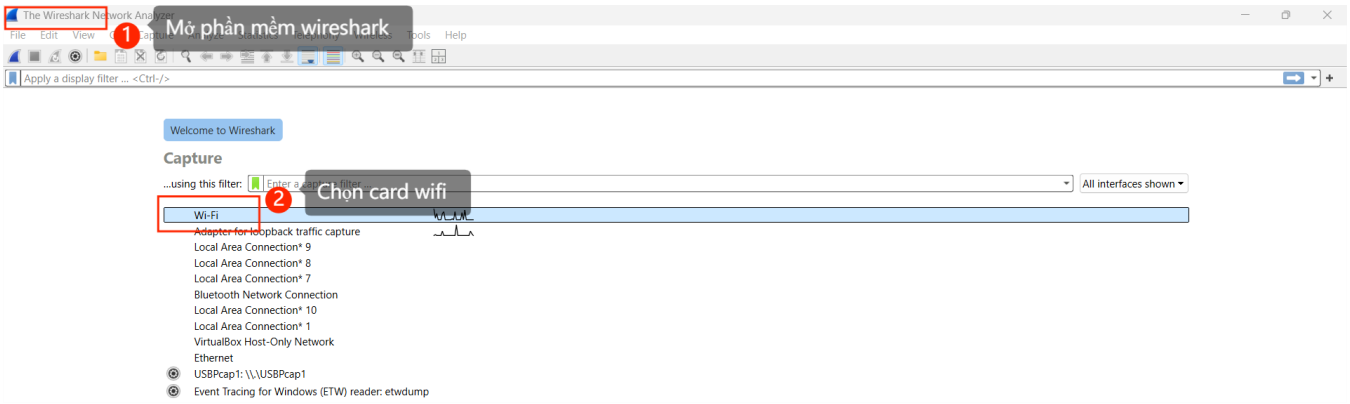
Hình 8.4. Tạo entity nhiệt độ, độ ẩm trên Home Assistant



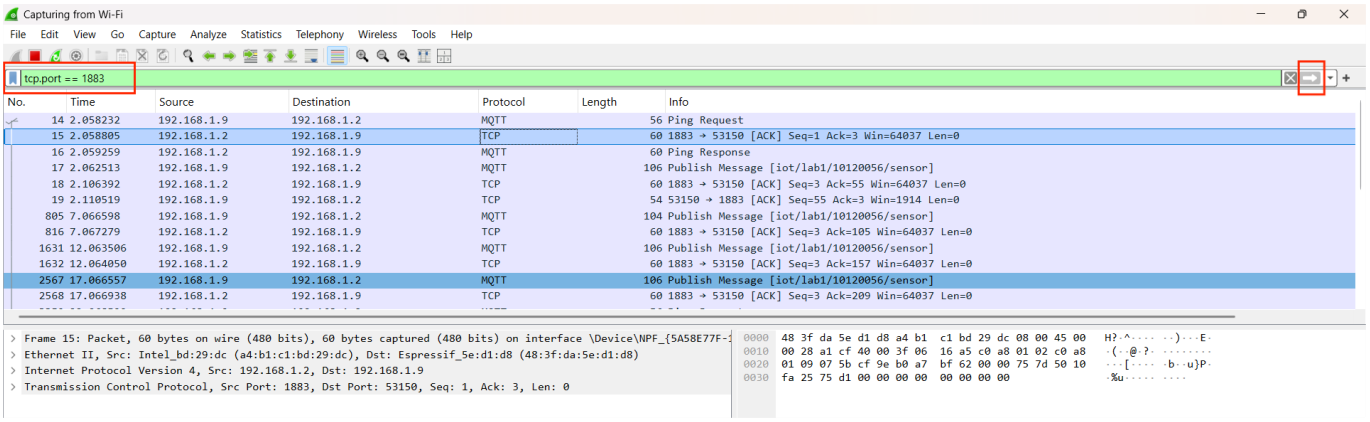
Hình 8.5. Giao diện hiển thị cảm biến nhiệt độ, độ ẩm trên Home Assistant

Bước 3 – Bắt gói Wireshark

- Lọc: **tcp.port == 1883**



Hình 8.6. Sử dụng phần mềm wireshark để bắt gói tin MQTT



Hình 8.7. Minh họa gói tin MQTT sử dụng phần mềm Wireshark

Kết luận & Trả lời câu hỏi

1. *Xác định luồng giao tiếp*

- Thiết bị nào là MQTT client / MQTT broker?
- Client / broker sử dụng cổng (port) nào?

2. *Phân tích mô hình TCP/IP*

- Layer 2 – Ethernet: Source MAC, Destination MAC, loại frame?
- Layer 3 – IP: Source IP, Destination IP, loại giao thức?
- Layer 4 – TCP: Source port / Destination port, Sequence number, Acknowledgment number, Vai trò của TCP đối với MQTT?

3. ***Nhận diện loại gói MQTT***
    - CONNECT
    - CONNACK
    - PUBLISH
    - PINGREQ
    - PINGRESP
    - DISCONNECT
4. ***Phân tích gói PUBLISH***
    - Topic name là gì?
    - QoS level?
    - Payload?
5. ***Keep Alive & Ping: Sinh viên quan sát Ping Request / Ping Response***
    - Mục đích của Ping là gì?
    - Nếu không có Ping thì điều gì xảy ra?
    - Ping diễn ra theo chu kỳ bao lâu?
6. ***Bảo mật***
    - Username/password MQTT có được mã hóa không?
    - Vì sao Wireshark đọc được payload?
    - Cần làm gì để tránh bị sniff dữ liệu?

Sinh viên thực hiện theo hướng dẫn và trả lời các câu hỏi

Bài 12.2. Điều khiển thiết bị IoT từ Home Assistant qua MQTT

Mục tiêu

Bài thực hành này yêu cầu sinh viên thiết kế và triển khai hệ thống điều khiển thiết bị IoT

trong đó topic MQTT đóng vai trò như một giao diện lập trình (API). Sinh viên thực hiện truyền thông hai chiều giữa ESP và Home Assistant thông qua các topic được thiết kế có cấu trúc rõ ràng, tách biệt lệnh điều khiển và trạng thái thiết bị.

Chuẩn bị

- *Phần cứng*
  - Module wifi ESP32/8266
  - 1 LED (hoặc LED onboard)
  - Cáp USB
- *Phần mềm*
  - Mosquitto broker
  - Home Assistant MQTT integration

Gợi ý thực hiện

Bước 1 – Thiết kế topic

- Sinh viên thiết kế topic

```
iot/lab2/<student_id>/cmd

iot/lab2/<student_id>/state
```

- Trong đó
  - cmd: Lệnh điều khiển
  - state: Trạng thái thiết bị

Bước 2 – ESP subscribe lệnh và publish trạng thái

```
// IMPORT IF ESP32

// #include <WiFi.h>
```

```
// IMPORT IF ESP8266

#include <ESP8266WiFi.h>

/* ===== CONFIG ===== */

const char* ssid     = "YOUR_WIFI_SSID";

const char* password = "YOUR_WIFI_PASSWORD";

const char* mqtt_server = "HA_IP_ADDRESS";

const int   mqtt_port   = 1883;

/* MQTT ACCOUNT (tạo trong Mosquitto HA) */

const char* mqtt_user = "MQTT_USERNAME";

const char* mqtt_pass = "MQTT_PASSWORD";

const char* student_id = "sv01";

/* MQTT TOPICS */

const char* topic_cmd   = "iot/lab2/sv01/cmd";

const char* topic_state = "iot/lab2/sv01/state";

/* ================== */

WiFiClient espClient;

PubSubClient client(espClient);

/* ESP32 onboard LED */

const int LED_PIN = 2;

bool ledState = false;
```

```
/* ==================== WIFI ==================== */

void setup_wifi() {

  Serial.print("Connecting to WiFi ");

  Serial.println(ssid);

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {

    delay(500);

    Serial.print(".");

  }

  Serial.println("\nWiFi connected");

  Serial.print("IP address: ");

  Serial.println(WiFi.localIP());

}

/* ==================== MQTT CALLBACK ==================== */

void callback(char* topic, byte* payload, unsigned int length) {

  Serial.print("Message arrived [");

  Serial.print(topic);

  Serial.print("] ");

  String message;
```

```
  for (unsigned int i = 0; i < length; i++) {
    message += (char)payload[i];
  }
  Serial.println(message);
  if (message == "ON") {
    ledState = true;
    digitalWrite(LED_PIN, HIGH);
  }
  else if (message == "OFF") {
    ledState = false;
    digitalWrite(LED_PIN, LOW);
  }
  /* Publish trạng thái LED */
  client.publish(topic_state, ledState ? "ON" : "OFF");
}
/* ================== MQTT RECONNECT ================== */
void reconnect() {
  while (!client.connected()) {
    String clientId = "ESP8266-Lab2-";
```

```
    clientId += student_id;
    Serial.print("Connecting to MQTT... ");
    if (client.connect(clientId.c_str(), mqtt_user, mqtt_pass)) {
      Serial.println("connected");
      client.subscribe(topic_cmd);
      Serial.print("Subscribed to ");
      Serial.println(topic_cmd);


    } else {
      Serial.print("failed, rc=");
      Serial.print(client.state());
      Serial.println(" retry in 2s");
      delay(2000);
    }
  }
}
/* ================== SETUP ================== */
void setup() {
  pinMode(LED_PIN, OUTPUT);
```

```
  digitalWrite(LED_PIN, LOW);
  Serial.begin(115200);
  setup_wifi();
  client.setServer(mqtt_server, mqtt_port);
  client.setCallback(callback);
  client.setKeepAlive(60);
}

/* ================== LOOP ================== */
void loop() {
  if (!client.connected()) {
    reconnect();
  }
  client.loop();
}
```

Bước 3 – Điều khiển từ HA

```
mqtt:
  switch:
    - name: "Lab2 SV01 LED"
      command_topic: "iot/lab2/sv01/cmd"
```

```
state_topic: "iot/lab2/sv01/state"
payload_on: "ON"
payload_off: "OFF"
optimistic: false
```

Sinh viên hoàn thành theo gợi ý trên.

Bài 12.3. Giữ trạng thái thiết bị IoT: Retain và bài toán "nhớ/quên"

Mục tiêu

Bài thực hành này giúp sinh viên hiểu cách quản lý trạng thái thiết bị IoT thông qua cơ chế retain của MQTT. Sinh viên quan sát sự khác biệt giữa việc có và không sử dụng retain message khi hệ thống hoặc thiết bị được khởi động lại. Qua đó, sinh viên phân biệt được trạng thái vật lý của thiết bị và trạng thái logic được lưu giữ trong hệ thống truyền thông IoT.

Chuẩn bị như bài tập 2

Gợi ý thực hiện

- Code ESP tham khảo

```
#if defined(ESP8266)
  #include <ESP8266WiFi.h>
#else
  #include <WiFi.h>
#endif

#include <PubSubClient.h>

/* ===== CONFIG ===== */
```

```
const char* ssid     = "YOUR_WIFI_SSID";
const char* password = "YOUR_WIFI_PASSWORD";

const char* mqtt_server = "HA_IP_ADDRESS";
const int   mqtt_port   = 1883;

/* MQTT account (tạo trong Mosquitto HA) */
const char* mqtt_user = "mqtt_user";
const char* mqtt_pass = "mqtt_pass";

const char* student_id = "sv01";
/* ================== */

WiFiClient espClient;
PubSubClient client(espClient);

/* ===== TOPIC (tự động theo student_id) ===== */
String topic_cmd;
String topic_state;
String topic_note;
String topic_lwt;
/* ========================================== */
```

```
#if defined(ESP8266)
const int LED_PIN = LED_BUILTIN;
#else
const int LED_PIN = 2;
#endif

bool ledState = false;
/* ===== LED CONTROL ===== */
void applyLed(bool on) {
  ledState = on;
#if defined(ESP8266)
  digitalWrite(LED_PIN, on ? LOW : HIGH);
#else
  digitalWrite(LED_PIN, on ? HIGH : LOW);
#endif
}

void publishStateRetained() {
  client.publish(topic_state.c_str(), ledState ? "ON" : "OFF", true);
}

/* ===== WIFI ===== */
```

```
void setup_wifi() {
  Serial.print("Connecting WiFi");
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(300);
    Serial.print(".");
  }
  Serial.println("\nWiFi connected");
}

/* ===== MQTT CALLBACK ===== */
void callback(char* topic, byte* payload, unsigned int length) {
  String msg;
  for (unsigned int i = 0; i < length; i++) msg += (char)payload[i];
  msg.trim();
  Serial.print("MQTT [");
  Serial.print(topic);
  Serial.print("] ");
  Serial.println(msg);

  if (String(topic) == topic_cmd) {
    if (msg == "ON")     applyLed(true);
```

```
    else if (msg == "OFF")    applyLed(false);
    else if (msg == "TOGGLE") applyLed(!ledState);
    publishStateRetained();
  }
}

/* ===== MQTT RECONNECT ===== */
void reconnect() {
  while (!client.connected()) {
    String clientId = "ESP-Lab3-";
    clientId += student_id;
    Serial.print("Connecting MQTT... ");
    if (client.connect(
          clientId.c_str(),
          mqtt_user,
          mqtt_pass,
          topic_lwt.c_str(),
          0,
          true,
          "OFFLINE"
        )) {
      Serial.println("connected");
```

```
      client.subscribe(topic_cmd.c_str());
      Serial.print("Subscribed ");
      Serial.println(topic_cmd);
      client.publish(topic_note.c_str(),
                     "ESP BOOT / RECONNECT (note not retained)",
                     false);

      client.publish(topic_lwt.c_str(), "ONLINE", true);
      publishStateRetained();
    } else {
      Serial.print("failed rc=");
      Serial.println(client.state());
      delay(1500);
    }
  }
}

/* ===== SETUP ===== */
void setup() {
  Serial.begin(115200);

  pinMode(LED_PIN, OUTPUT);
```

```
  applyLed(false);


  /* Build topics */
  topic_cmd   = "iot/lab3/" + String(student_id) + "/cmd";
  topic_state = "iot/lab3/" + String(student_id) + "/state";
  topic_note  = "iot/lab3/" + String(student_id) + "/note";
  topic_lwt   = "iot/lab3/" + String(student_id) + "/lwt";
  setup_wifi();
  client.setServer(mqtt_server, mqtt_port);
  client.setCallback(callback);
  client.setKeepAlive(60);
}


/* ===== LOOP ===== */
void loop() {
  if (!client.connected()) reconnect();
  client.loop();
}
```

- Cấu hình HA tham khảo

```
mqtt:

  switch:
```

```
- name: "Lab3 SV01 LED (Retain State)"
  command_topic: "iot/lab3/sv01/cmd"
  state_topic: "iot/lab3/sv01/state"
  payload_on: "ON"
  payload_off: "OFF"
  optimistic: false
```

Câu hỏi kiểm tra

- Khi restart Home Assistant, switch có hiện đúng trạng thái LED không? Vì sao?
- Khi **restart ESP**, trạng thái HA thay đổi như thế nào?
- Nếu **retain = false**, chuyện gì khác?
- Retain có phải là "lưu dữ liệu lâu dài" không? Nó khác database ở đâu?

Sinh viên hoàn thành theo gợi ý và trả lời câu hỏi trên.

HẾT